Hacettepe University

Computer Engineering Department

BBM204 Software Practicum II - 2025 Spring

Programming Assignment 1

March 21, 2025

Student name:
Mustafa Emir TEZCAN

Student Number:
b2230356092

1 Problem Definition

The goal of this experiment is to analyze and compare the performance of various sorting algorithms by measuring their running times across datasets of different types (e.g., sorted, reverse sorted, random) and sizes. By evaluating these algorithms, we aim to determine their efficiency in handling diverse input characteristics and to determine whether the obtained results (running times of our algorithm implementations) agree with their theoretical asymptotic complexities.

2 Solution Implementation

Comparison-based sorting (Comb sort, Insertion sort, Shaker sort, Shell sort) and Non-comparison-based sorting algorithms (Radix sort) were implemented in this experiment.All algorithms were run 10 times for 10 different input sizes (n) from 500 to 250,000 and 3 different data types (random, sorted, reversely sorted), the times were recorded and the average times (ms) were calculated.

2.1 Sorting Algorithm 1: Comb Sort

```java
public class CombSort {
    public  static void sort(int[] array){
        int gap = array.length;
        double shrink = 1.3;
        boolean sorted=false;
        while(!sorted){
            gap= Math.max(1,(int) Math.floor(gap / shrink));
            sorted =(gap==1);
            for (int i=0; i< array.length-gap; i++){
                if(array[i]>array[i+gap]){
                    int temp = array[i];
                    array[i]=array[i+gap];
                    array[i+gap]=temp;
                    sorted=false;
                }
            }
        }
    }
}
```

In this algorithm, elements are compared with a certain integer number (”gap”, array length) and this gap is narrowed by the ”shrink” factor. A shrink factor of 1.3 is ideal for efficiency.

2.2 Sorting Algorithm 2: Insertion Sort

```java
public class InsertionSort {
    public static void sort(int[] array){
        for (int j=1;j<array.length;j++){
            int key = array[j];
            int i = j-1;
            while (i>=0&&array[i]>key){
                array[i+1]=array[i];
                i--;
            }
            array[i+1]=key;
        }
    }
}
```

In this algorithm, each element is compared with the previous elements from its current position and placed in the appropriate position until all the data is sorted. The elements are swapped to the right to form a sorted array by shifting the larger elements to the right.

2.3 Sorting Algorithm 3: Shaker Sort

```java
public class ShakerSort {
    public static void sort(int[] array){
        boolean swapped = true;
        while (swapped){
            swapped = false;
            for(int i=0;i<array.length-1;i++){
                if (array[i]>array[i+1]){
                    int temp= array[i];
                    array[i]=array[i+1];
                    array[i+1]=temp;
                    swapped=true;
                }
            }
            if(!swapped){
                break;
            }
            swapped = false;
            for (int i=array.length-2;i>=0;i--){
                if (array[i]>array[i+1]){
                    int temp= array[i];
                    array[i]=array[i+1];
                    array[i+1]=temp;
                    swapped=true;
                }
            }
```

```
        }
    }
}
```

In this algorithm, two-way comparisons are made from one end of the array to the other and back again until the data is sorted. In each pass, larger elements are moved to the end and smaller elements to the beginning, respectively. Thus, both the minimum and maximum elements are placed in their correct positions.

2.4 Sorting Algorithm 4: Shell Sort

```
public class ShellSort {
    public static void sort(int[] array){
        int n= array.length;
        int gap= n/2;
        while (gap>0){
            for(int i=gap;i<n;i++){
                int temp= array[i];
                int j=i;
                while (j>=gap&&array[j-gap]>temp){
                    array[j]=array[j-gap];
                    j=j-gap;
                }
                array[j]=temp;
            }
        gap=gap/2;
        }
    }
}
```

In this algorithm, elements are compared at certain intervals (gap) to speed up sorting. A wider interval is used at first, and this interval is reduced (1/2) at each step, allowing elements to be compared with their closer neighbors.

2.5 Sorting Algorithm 5: Radix Sort

```
public class RadixSort {
    public static int[] sort(int[] array,int d) {
        for(int pos=1; pos<=d; pos++){
            array= countingSort(array, pos);
        }
        return array;
    }

    public static int[] countingSort(int[] array, int pos) {
        int[] count = new int[10];
```

```java
        int[] output = new int[array.length];
        int size = array.length;

        for (int i = 0; i < size; i++) {
            int digit = getDigit(array[i], pos);
            count[digit]++;
        }

        for (int i = 1; i < 10; i++) {
            count[i] += count[i - 1];
        }

        for (int i = size - 1; i >= 0; i--) {
            int digit = getDigit(array[i], pos);
            count[digit]--;
            output[count[digit]] = array[i];
        }

        return output;
    }
    public static int getDigit(int number, int pos) {
        return (number / (int) Math.pow(10, pos - 1)) % 10;
    }
}
```

In this algorithm, elements are sorted according to their digits. Starting from the lowest significant digit (LSD), each digit is considered in order. Elements are grouped according to their digit values and the sorting process is repeated between digits.

3 Results, Analysis, Discussion

The average running times (in milliseconds) for all algorithms, across 10 different input sizes (n) (from 500 to 250,000) and 3 different data types (random, sorted, and reversely sorted), are given in Table 1.

Complexity analysis tables are given. (Table 2 and Table 3):

" What are the best, average, and worst cases for the given algorithms in terms of the given input data to be sorted? " and " Do the obtained results (running times of your algorithm implementations) match their theoretical asymptotic complexities? " questions were analyzed for each graph, considering the all graphs and tables.

Table 1: Results of the running time tests performed for varying input sizes (in ms).

	Input Size n									
Algorithm	**500**	**1000**	**2000**	**4000**	**8000**	**16000**	**32000**	**64000**	**128000**	**250000**
	Random Input Data Timing Results in ms									
Comb sort	0.05	0.08	0.04	0.25	0.40	0.65	1.43	3.44	6.72	19.39
Insertion sort	0.09	0.09	0.22	1.34	3.36	17.32	49.87	192.29	854.57	3514.52
Shaker sort	0.08	0.64	0.92	3.30	14.89	55.76	184.05	760.91	5819.96	34451.56
Shell sort	0.04	0.19	0.06	0.19	0.58	1.01	2.26	6.08	11.95	31.14
Radix sort	0.15	0.28	0.46	1.50	1.95	3.47	6.86	15.56	27.67	86.30
	Sorted Input Data Timing Results in ms									
Comb sort	0.01	0.01	0.02	0.04	0.13	0.22	0.35	0.82	1.56	4.17
Insertion sort	0.00	0.00	0.01	0.01	0.01	0.02	0.04	0.11	0.24	0.38
Shaker sort	0.00	0.00	0.00	0.00	0.01	0.01	0.02	0.06	0.11	0.27
Shell sort	0.01	0.01	0.02	0.04	0.10	0.18	0.41	0.92	1.92	4.08
Radix sort	0.14	0.29	0.68	1.21	0.13	4.77	10.10	19.22	39.69	76.43
	Reversely Sorted Input Data Timing Results in ms									
Comb sort	0.01	0.02	0.03	0.09	0.15	0.22	0.34	0.74	1.56	4.68
Insertion sort	0.07	0.17	0.74	2.77	12.49	43.53	130.85	389.96	1545.62	7530.28
Shaker sort	0.22	0.59	2.21	9.31	37.82	124.07	337.18	1349.82	5396.45	28755.07
Shell sort	0.01	0.02	0.03	0.11	0.23	0.26	0.40	0.86	1.89	5.46
Radix sort	0.16	0.43	0.62	1.47	2.85	4.64	7.04	12.49	27.95	78.92

Table 2: Computational complexity comparison of the given algorithms.

Algorithm	**Best Case**	**Average Case**	**Worst Case**
Comb sort	$\Omega(nlogn)$	$\Theta(n^2/2^p)$	$O(n^2)$
Insertion sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$
Shaker sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$
Shell sort	$\Omega(nlogn)$	$\Theta(nlog^2n)$	$O(n^2)$
Radix sort	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$

Table 3: Auxiliary space complexity of the given algorithms.

Algorithm	**Auxiliary Space Complexity**
Comb sort	$O(1)$
Insertion sort	$O(1)$
Shaker sort	$O(1)$
Shell sort	$O(1)$
Radix sort	$O(n+k)$

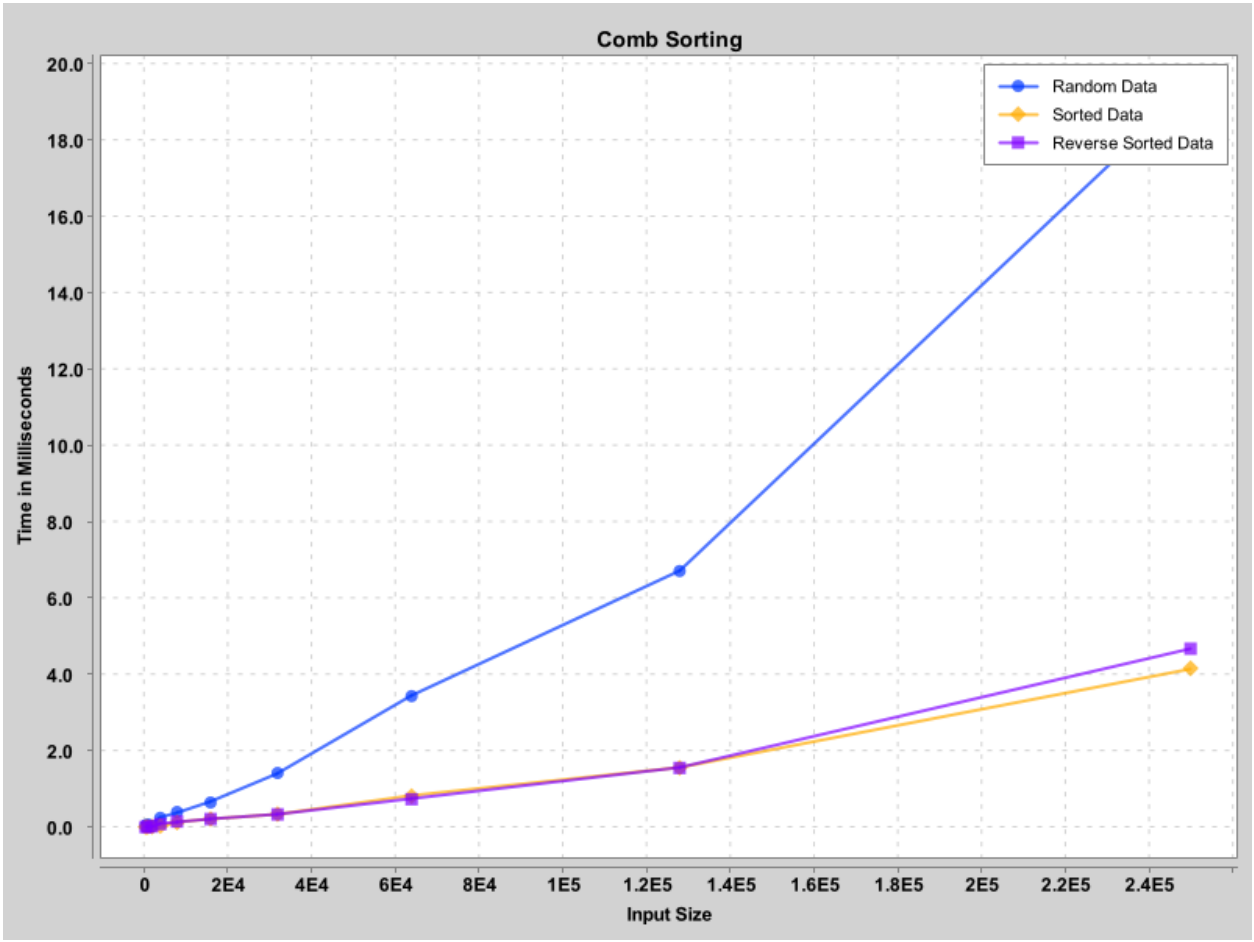


Figure 1: Comb sort algorithm

According to Table 2, the best case time complexity of Comb Sort algorithm is determined as $O(nlogn)$. In addition, Figure 1 shows the sorted input data as the type that results in the shortest running time for Comb Sort. The plotted graph for sorted data shows almost linear properties and confirms the calculated time complexity in Table 2. In the worst case random data verifies the time complexity O(n^2) and draws a graph similar to a parabola.

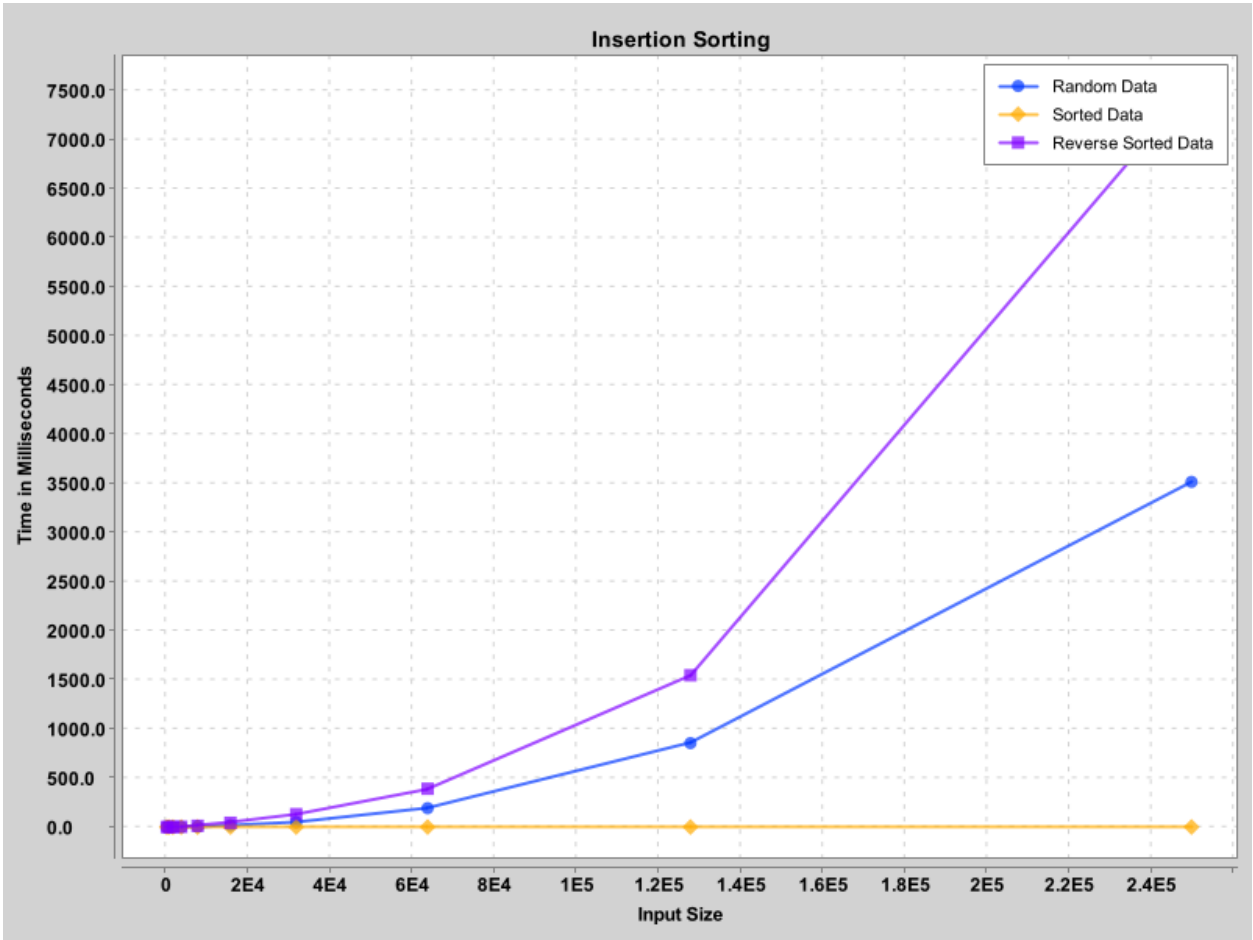


Figure 2: Insertion sort algorithm

According to Table 2, the best case time complexity of Insertion Sort algorithm is determined as $O(n)$. In addition, Figure 2 shows the sorted input data as the type that results in the shortest running time for Insertion Sort . The plotted graph for sorted data shows linear properties(almost 0) and confirms the calculated time complexity in Table 2. The time complexity in the worst and average cases is $O(n^2)$ and a similar to a parabola is plotted for reversely sorted and random inputs.

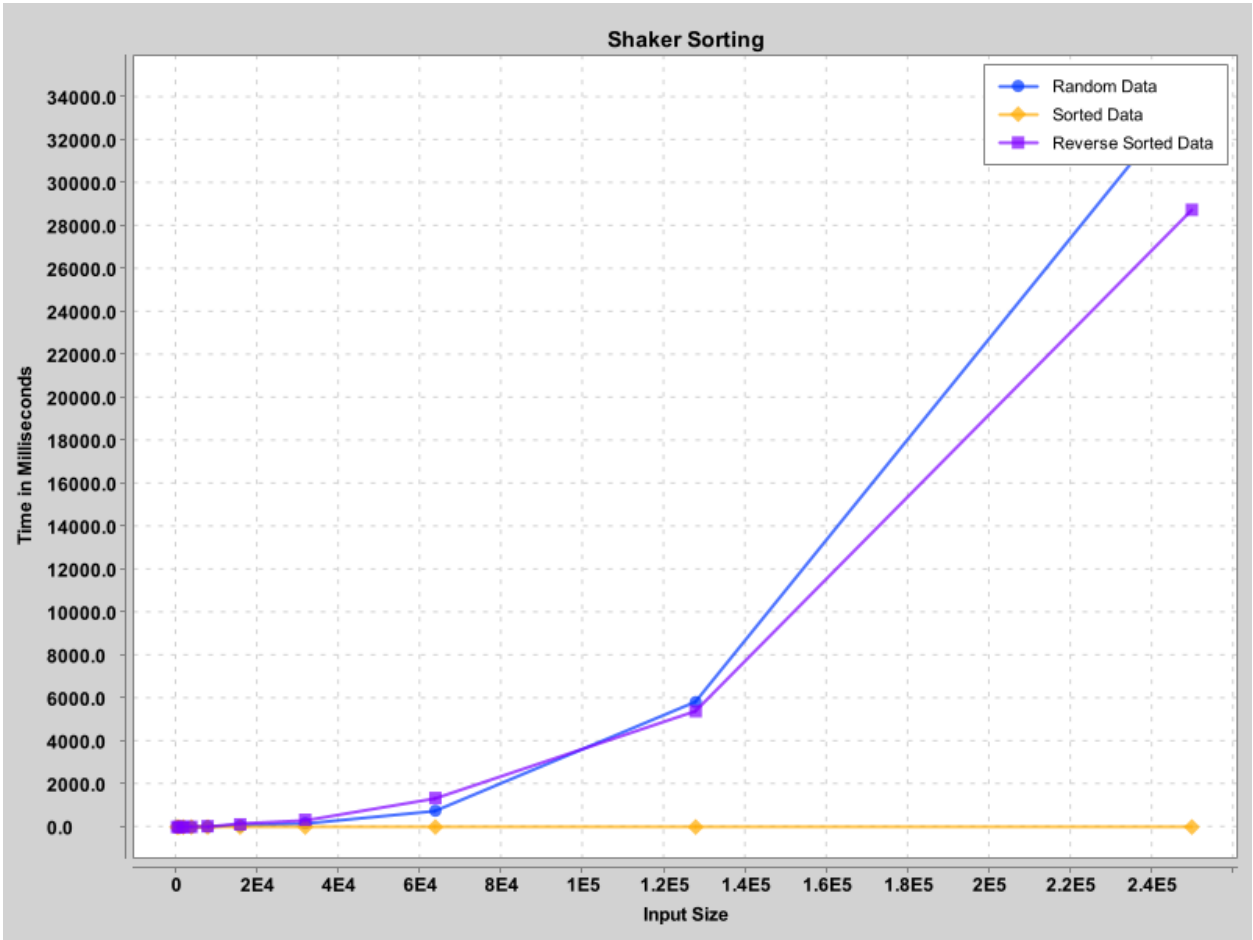


Figure 3: Shaker sort algorithm

According to Table 2, the best case time complexity of Shaker Sort algorithm is determined as $O(n)$. In addition, Figure 3 shows the sorted input data as the type that results in the shortest running time for Shaker Sort . The plotted graph for sorted data shows linear properties(almost 0) and confirms the calculated time complexity in Table 2. The time complexity in the worst and average cases is $O(n^2)$ and a similar to a parabola is plotted for random and reversely sorted inputs.

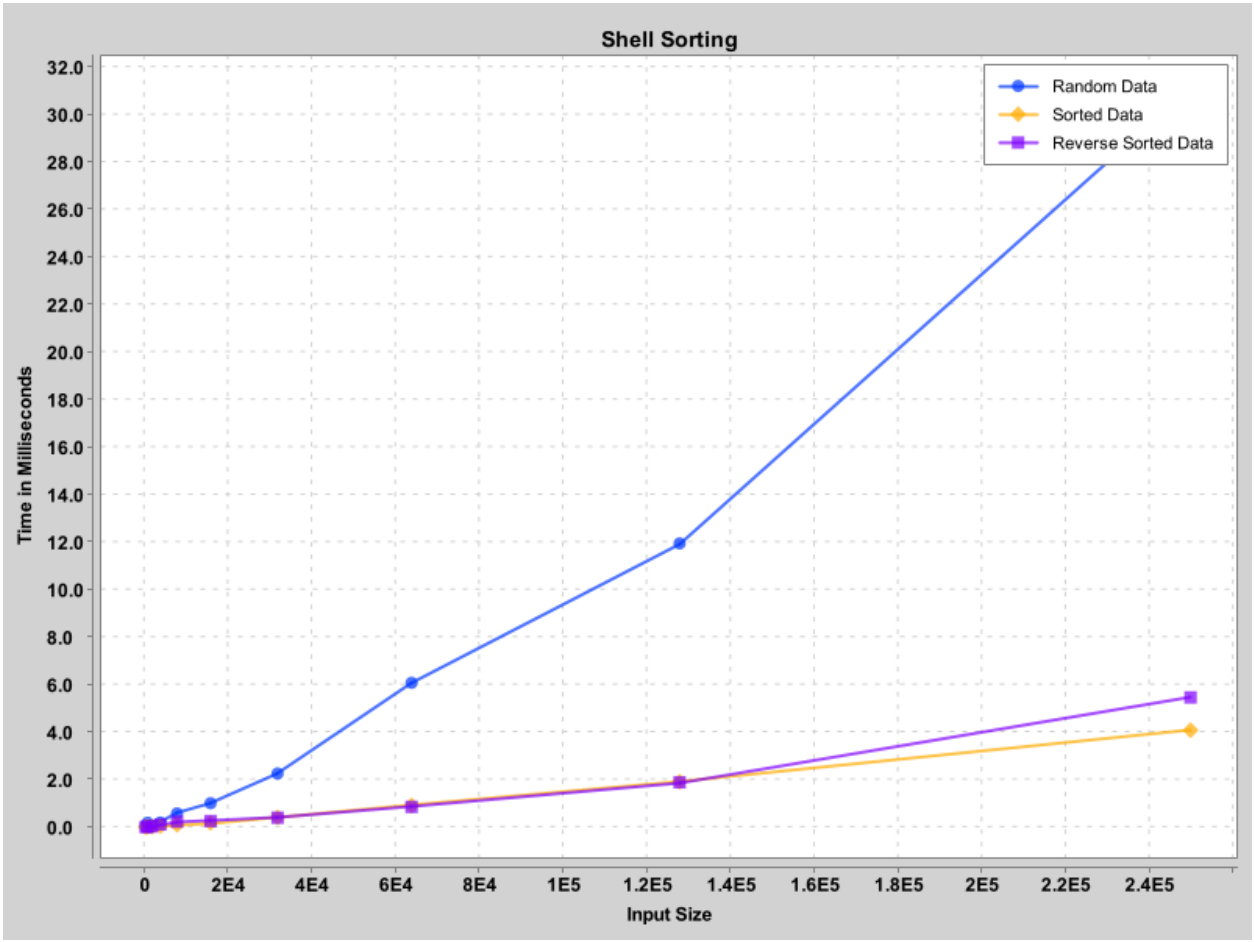


Figure 4: Shell sort algorithm

According to Table 2, the best case time complexity of Shell Sort algorithm is determined as $O(nlogn)$. In addition, Figure 4 shows the sorted input data as the type that results in the shortest running time for Shell Sort . This sorted input curve is similar to the curve of Comb Sort and in the best case, the same time complexity calculations will verify both graphs. The time complexity in the worst case is $O(n^2)$ and a similar to a parabola is plotted for random input.

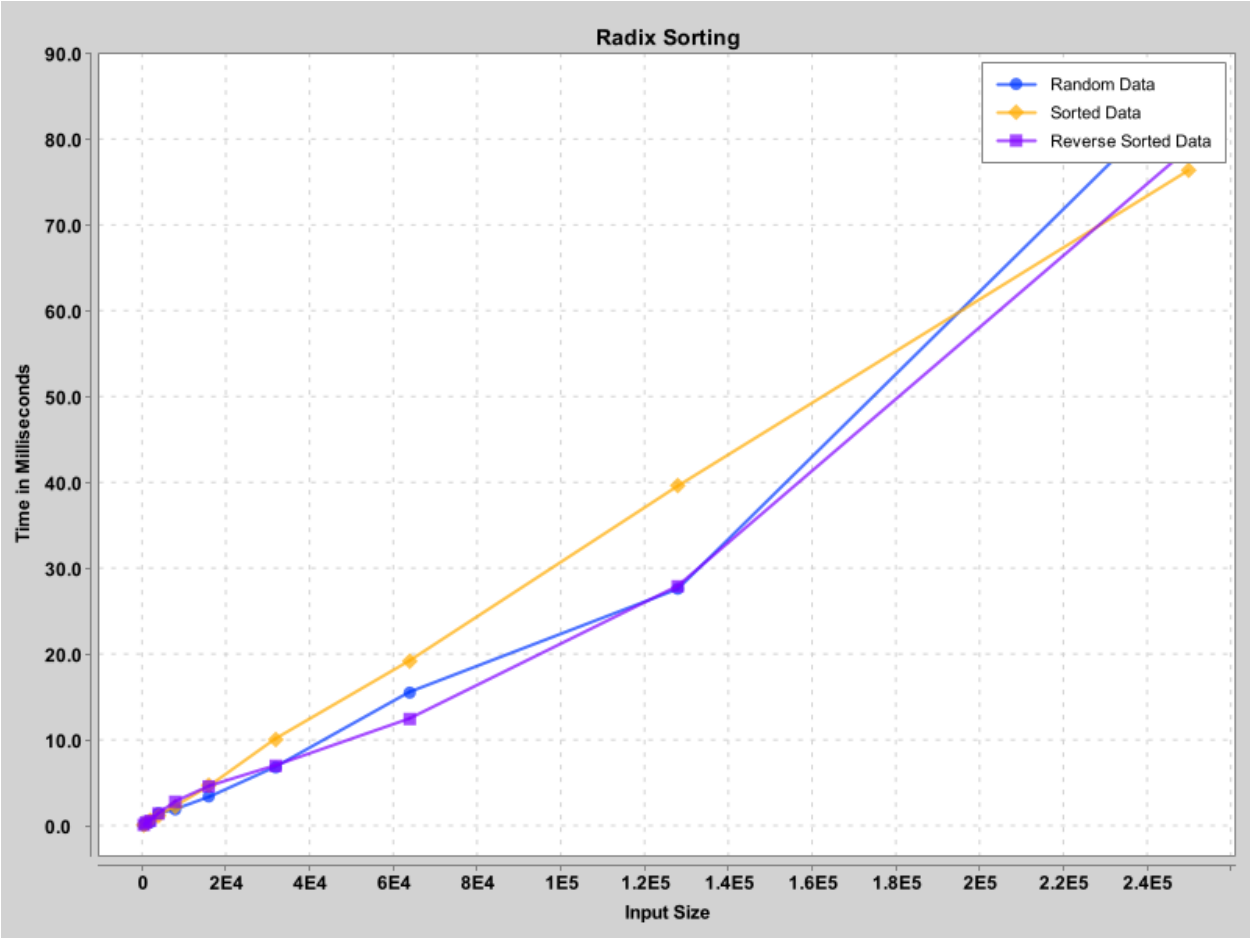
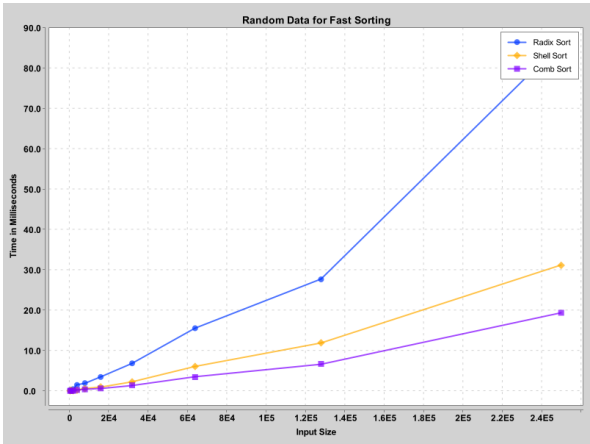
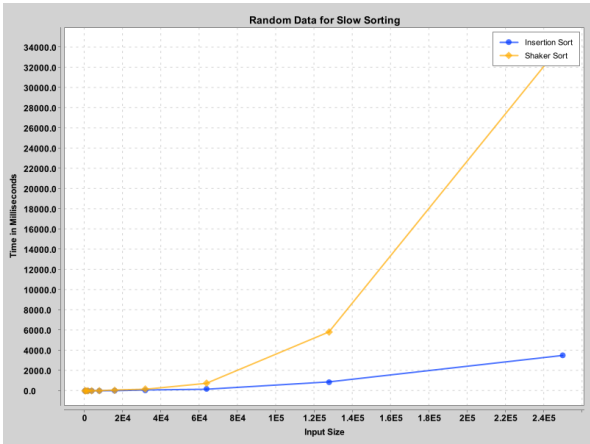


Figure 5: Radix sort algorithm

According to Table 2, the best, average and worst case time complexity of the Radix Sort algorithm is calculated to be $O(nk)$. Additionally, Figure 5 shows that very similar to each other near-linear curves are generated for all input data types. This plot again confirms the calculated time complexities.mIn addition, up to 250,000 input size the best case is variable(reversely sorted or random), after that the sorted data is in the best case state.



(a) Random Data for Fast Sorting Algorithms

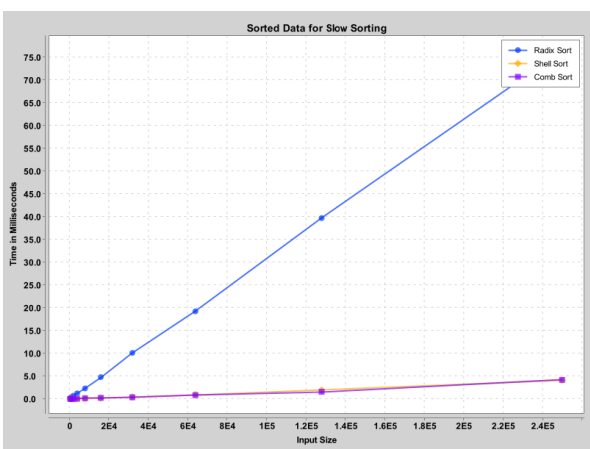


(b) Random Data for Slow Sorting Algorithms

Figure 6: Comparison of all sorting algorithms in Random Data

The fastest algorithm for random data is Comb sort, while the slowest is shaker sort. In addition to these except for radix sort $O(nk)$, other sorting types have $O(n^2)$ time complexity for random data(worst case except insertion). This is also confirmed in the calculated theoretical asymptotic complexity figures 6a and 6b.

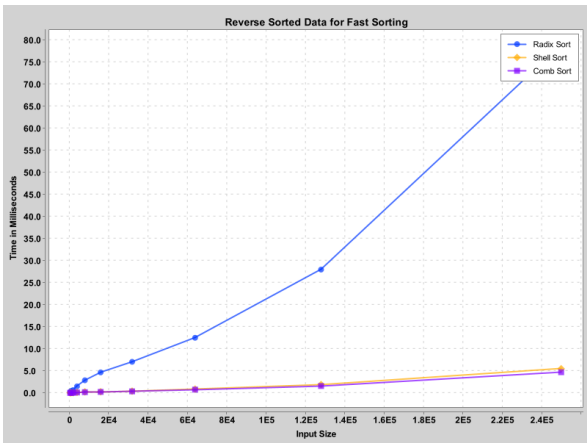
(a) Sorted Data for Fast Sorting Algorithms



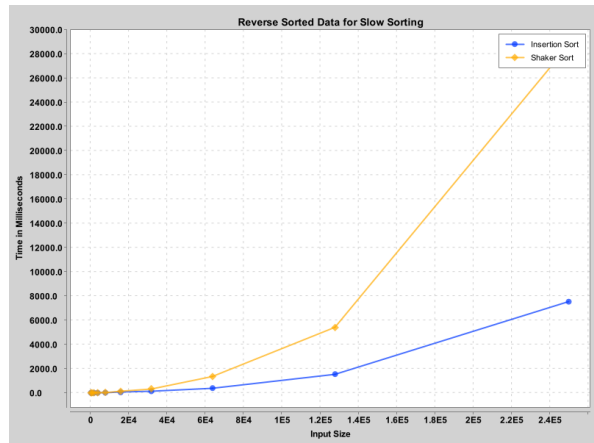
(b) Sorted Data for Slow Sorting Algorithms

Figure 7: Comparison of all sorting algorithms in Sorted Data

The fastest algorithm for sorted data is Shaker sort, while the slowest is Radix sort. For all algorithms, the best case is generally sorted data at any input size.Since they have $O(logn)$, $O(n)$ and $O(nk)$ time complexities in the best case, they should draw lines close to linearity. The figures 7a and 7b also confirm this.



(a) Reversely Sorted Data for Fast Sorting Algorithms



(b) Reversely Sorted Data for Slow Sorting Algorithms

Figure 8: Comparison of all sorting algorithms in Reversely Sorted Data

The fastest algorithm for sorted data is Comb sort, while the slowest is Shaker sort. The average case of all algorithms except insertion sort is reversely sorted data for all input sizes. Since they have $O(n^2)$, and time complexities , Insertion sort and Shaker sort should draw lines similar to parabola. On the other hand, since they have $O(nk)$, $O(nlog^2n)$ and $O(n^2/2^p)$ time complexities , other sorting algorithms should draw lines close to linearity. The figures 8a and 8b also confirm this.

4 Some Questions and Additional Explanations

1-) Do Shaker Sort and Comb Sort improve performance compared to Bubble Sort? If so,how do their approaches contribute to this improvement?

Shaker Sort and Comb Sort improve performance compared to Bubble Sort and they do so through their distinct approaches that address Bubble Sort's inefficiencies. Shaker Sort improves upon Bubble Sort by sorting in both directions during a pass through the list. In Bubble Sort, after a single pass, the largest element is moved to its correct position. However, the smallest element might still be far from its correct place. On the other hand, Shaker sort bubbles the largest element all the way in one direction, then sweeps it back in the opposite direction and moves the smallest element to its correct position. This two-way approach makes each pass more efficient and reduces the number of passes.

Comb Sort improves performance by addressing one of Bubble Sort's biggest drawbacks: comparing and swapping only adjacent elements. It introduces a "gap" between compared elements, which starts large and shrinks over time, allowing for more efficient comparison and movement of elements. The gap size is progressively reduced (usually divided by a factor of 1.3) until it becomes 1, at which point Comb Sort behaves similarly to Bubble Sort but with far fewer unnecessary comparisons earlier on.

2-) Is Shell Sort perform better than Insertion Sort for larger datasets? Under what conditions does Insertion Sort still perform well?

Shell Sort typically outperforms Insertion Sort for larger datasets. It improves on Insertion Sort by allowing comparisons of elements that are farther apart using a "gap" sequence. This efficiently moves elements closer to their final positions early on, reducing the number of overall comparisons and shifts as the gap decreases. This makes Shell Sort better suited for larger datasets. However Insertion Sort can still perform well under certain conditions like already sorted or nearly sorted datas and small datasets.

3-) Given that Radix Sort is a non-comparison-based sorting algorithm, how does it handle large numerical ranges efficiently ?

Radix Sort handles large numerical ranges efficiently by breaking numbers down into their individual digits and sorting based on these, one digit at a time. Instead of comparing entire numbers, Radix Sort processes digits starting from the least significant digit (LSD) to the most significant digit (MSD) .The number of passes depends on the number of digits in the largest number (e.g. a 10-digit number requires 10 passes). This ensures predictable performance. Additionaly, Radix Sort focuses on the digits of numbers rather than their magnitude or range. Thus, even if the numerical range is large, it performs well as long as the number of digits isn't excessively high.

References

- PROGRAMMING ASSIGNMENT 1.pdf